

Sorting Algorithms

Why Study Sorting Algorithms?

- There are a variety of situations that we can encounter
 - Do we have randomly ordered keys?
 - Are all keys distinct?
 - How large is the set of keys to be ordered?
 - Need guaranteed performance?
- Various algorithms are better suited to some of these situations

Some Definitions

- Internal Sort
 - The data to be sorted is all stored in the computer's main memory.
- External Sort
 - Some of the data to be sorted might be stored in some external, slower, device.
- In Place Sort
 - The amount of extra space required to sort the data is constant with the input size.

Stability

- A **STABLE** sort preserves relative order of records with equal keys

Sorted on first key:

Aaron	4	A	664-480-0023	097 Little
Andrews	3	A	874-088-1212	121 Whitman
Battle	4	C	991-878-4944	308 Blair
Chen	2	A	884-232-5341	11 Dickinson
Fox	1	A	243-456-9091	101 Brown
Furia	3	A	766-093-9873	22 Brown
Gazsi	4	B	665-303-0266	113 Walker
Kanaga	3	B	898-122-9643	343 Forbes
Rohde	3	A	232-343-5555	115 Holder
Quilici	1	C	343-987-5642	32 McCosh

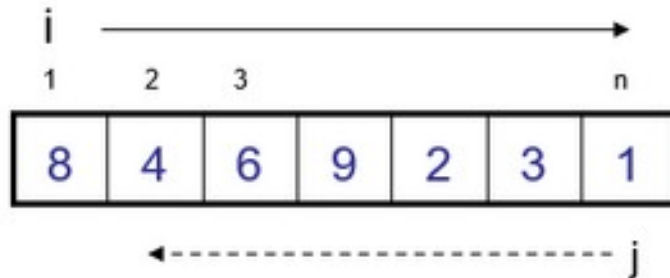
Sort file on second key:

Fox	1	A	243-456-9091	101 Brown
Quilici	1	C	343-987-5642	32 McCosh
Chen	2	A	884-232-5341	11 Dickinson
Kanaga	3	B	898-122-9643	343 Forbes
Andrews	3	A	874-088-1212	121 Whitman
Furia	3	A	766-093-9873	22 Brown
Rohde	3	A	232-343-5555	115 Holder
Battle	4	C	991-878-4944	308 Blair
Gazsi	4	B	665-303-0266	113 Walker
Aaron	4	A	664-480-0023	097 Little

Records with key value
3 are not in order on
first key!!

Bubble Sort

- Idea:
 - Repeatedly pass through the array
 - Swaps adjacent elements that are out of order



- Easier to implement, but slower

Bubble Sort

Algorithm 1: Bubble sort

Data: Input array $A[]$

Result: Sorted $A[]$

int i, j, k ;

$N = \text{length}(A)$;

for $j = 1$ **to** N **do**

for $i = 0$ **to** $N-1$ **do**

if $A[i] > A[i+1]$ **then**

$\text{temp} = A[i]$;

$A[i] = A[i+1]$;

$A[i+1] = \text{temp}$;

end

end

end

Bubble Sort

Algorithm 2: Improved Bubble Sort

Data: Input array $A[]$

Result: Sorted $A[]$

$int\ i, j, k;$

$indicator = 1;$

$N = length(A);$

for $j = 1; j \leq (N-1) \wedge indicator == 1; j++$ **do**

$indicator = 0;$

for $i = 0$ to $N-1; i++$ **do**

if $A[i] > A[i+1]$ **then**

$temp = A[i];$

$A[i] = A[i+1];$

$A[i+1] = temp;$

$indicator = 1;$

end

end

end

```
% Bubble sort an array of elements A
n = length(A)
swapped = true
while( swapped )
    swapped = false
    for(j = 0; j < n-1; j++)
        if( A[j] > A[j+1] )
            swap(A[j], A[j+1])
            swapped = true
    n = n-1
```


Bubble Sort - Analysis

- Worst case time: $O(n^2)$
- Best case time: $O(n)$ (array is already sorted)
- Stable: equal items stay in the same order
- An *in place* algorithm (output array is same as input array, uses at most a small, constant amount of additional space)

Selection Sort

- Idea:
 - Find the smallest element in the array
 - Exchange it with the element in the first position
 - Find the second smallest element and exchange it with the element in the second position
 - Continue until the array is sorted
- Disadvantage:
 - Running time depends only slightly on the amount of order in the file

Selection Sort

Alg.: SELECTION-SORT(A)

$n \leftarrow \text{length}[A]$

for $j \leftarrow 0$ **to** $n - 1$

do $\text{smallest} \leftarrow j$

for $i \leftarrow j + 1$ **to** n

do if $A[i] < A[\text{smallest}]$

then $\text{smallest} \leftarrow i$

 exchange $A[j] \leftrightarrow A[\text{smallest}]$

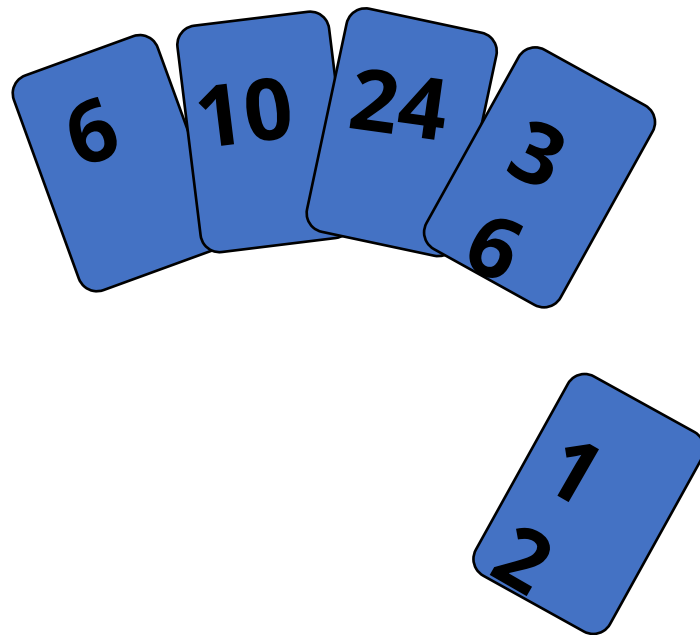
Selection Sort

- Worst case time: $O(n^2)$
 - Best case time: same as worst case
 - Maximum number of swaps: $n - 1$
 - Stable and in place.
- Mostly stable, but for cases like array containing **2, 2, 1, or 4, 2, 4, 1** it is not stable

Insertion Sort

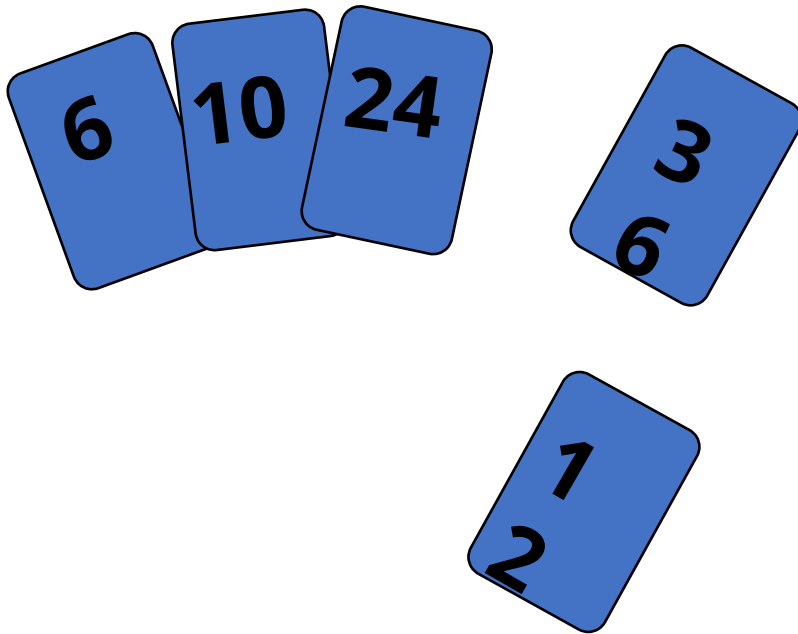
- Idea: like sorting a hand of playing cards
 - Start with an empty left hand and the cards facing down on the table.
 - Remove one card at a time from the table, and insert it into the correct position in the left hand
 - compare it with each of the cards already in the hand, from right to left
 - The cards held in the left hand are sorted
 - these cards were originally the top cards of the pile on the table

Insertion Sort

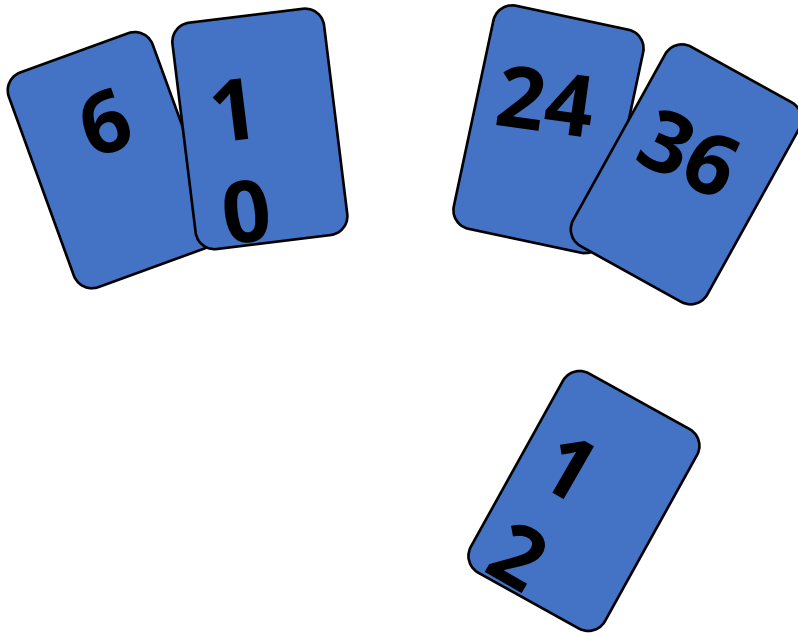


To insert 12, we need to make room for it by moving first 36 and then 24.

Insertion Sort



Insertion Sort



Insertion Sort

Insertion Sort

Alg.: INSERTION-SORT(A)

for $j \leftarrow 1$ **to** n

do $key \leftarrow A[j]$

Insert $A[j]$ into the sorted sequence $A[1..j-1]$

$i \leftarrow j - 1$

while $i \geq 0$ and $A[i] > key$

do $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow key$

- Insertion sort – sorts the elements in place

Insertion Sort

- Worst case time: $O(n^2)$ (array is in reverse order)
- Best case time: $O(n)$ (array is already sorted)
- Maximum number of swaps: $n - 1$
- Stable and in place.
- If each item starts no more than k positions from its sorted position, then the time is $k \cdot O(n)$ or $O(n)$ (why?)

$$k.O(n)$$

Heap Sort

Heap Sort Algorithm for sorting in increasing order:

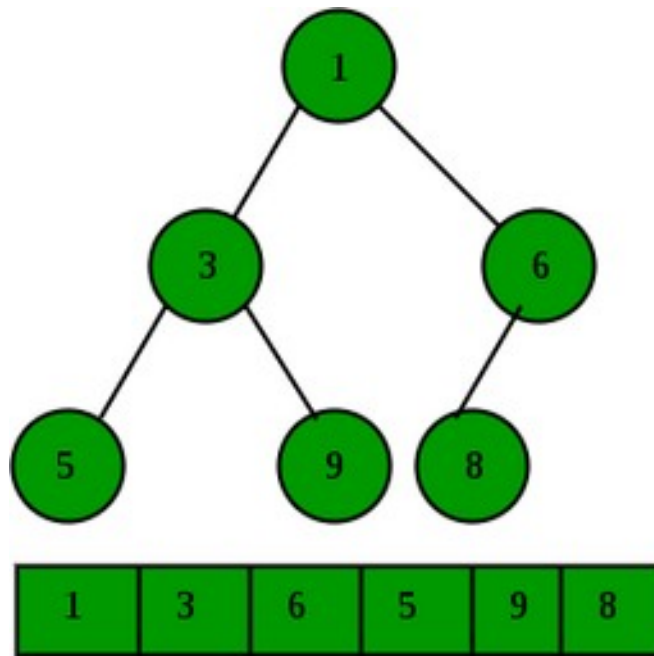
Step 1: Build a max heap from the input data.

Step 2: At this point, the largest item is stored at the root of the heap.

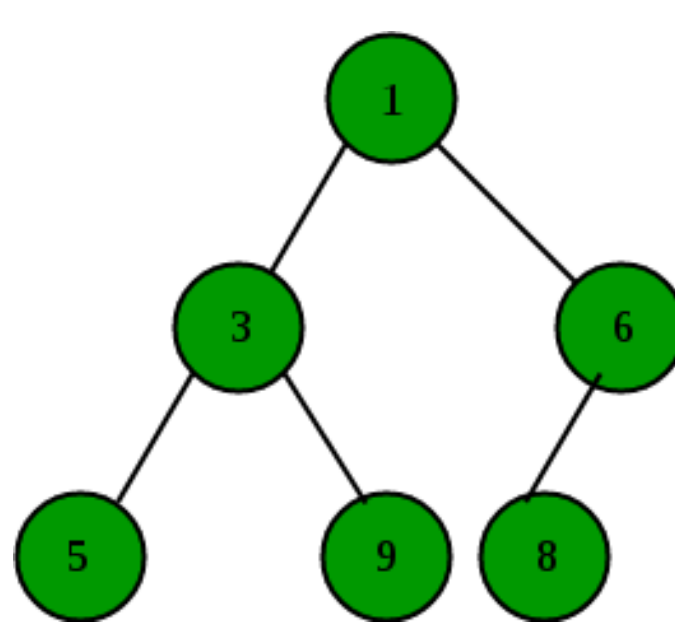
Replace it with the last item of the heap followed by reducing

the size of heap by 1. Finally, heapify the root of the tree.

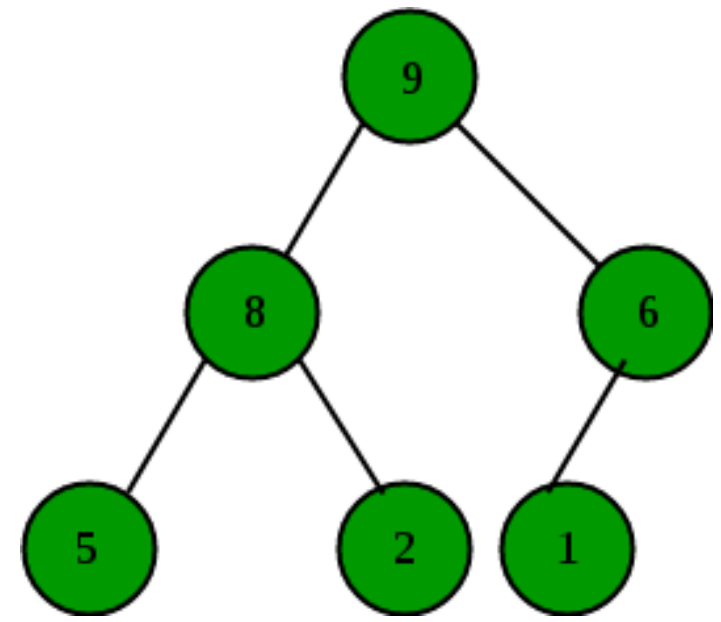
Heap Sort



Input Array
Heap

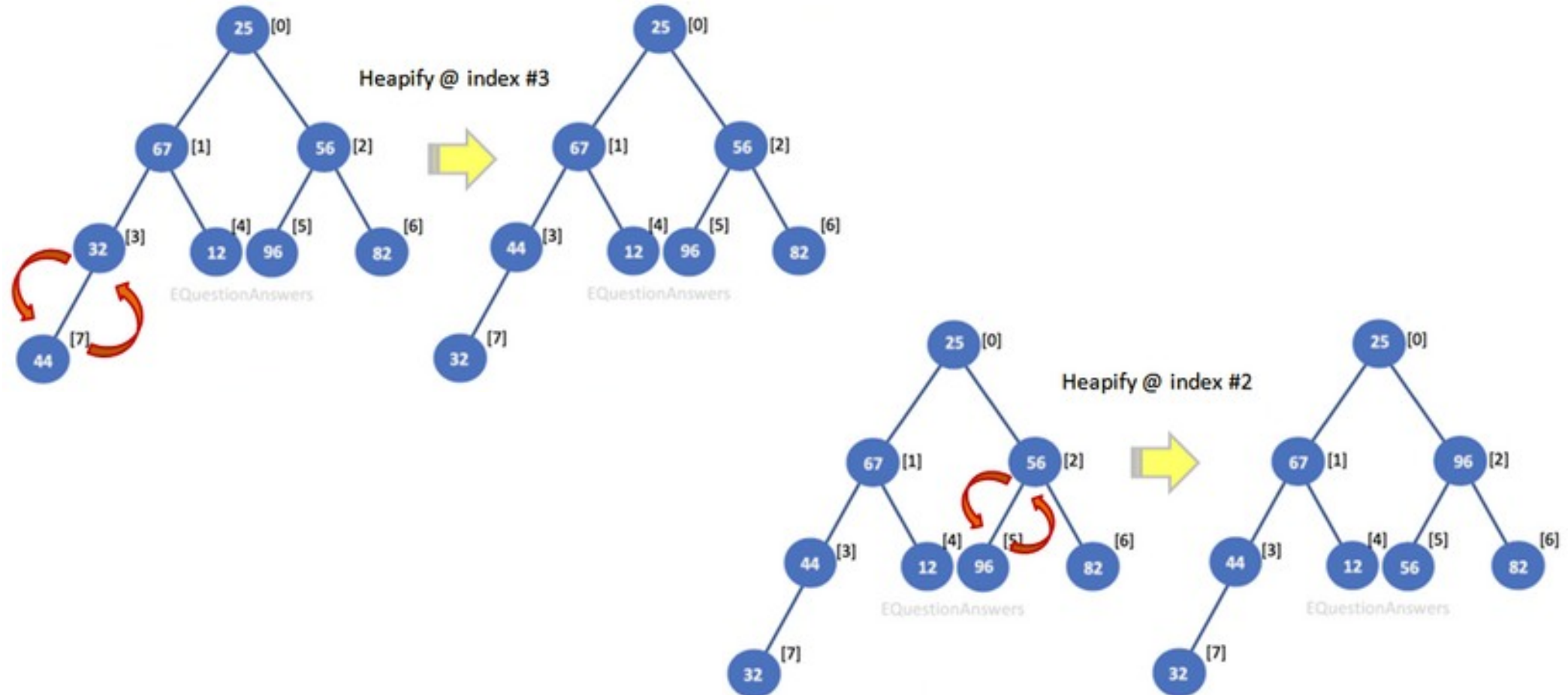


Min Heap

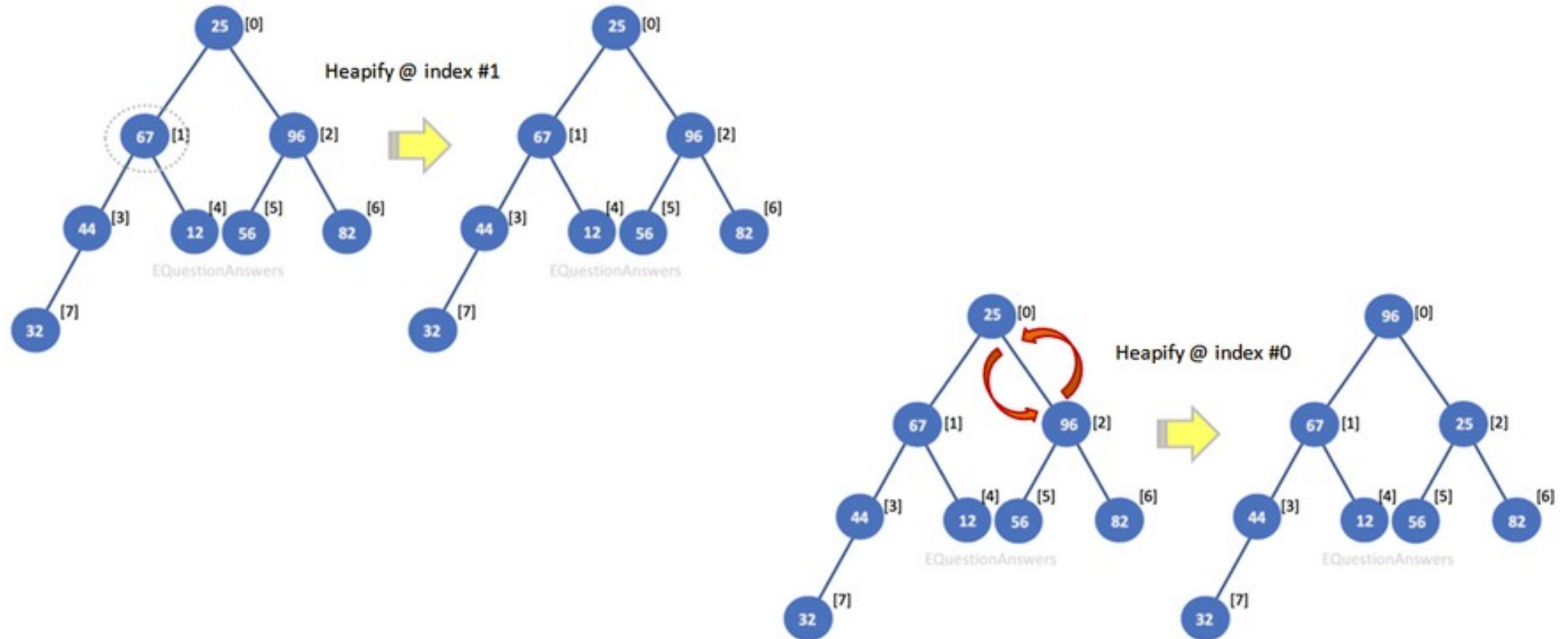


Max

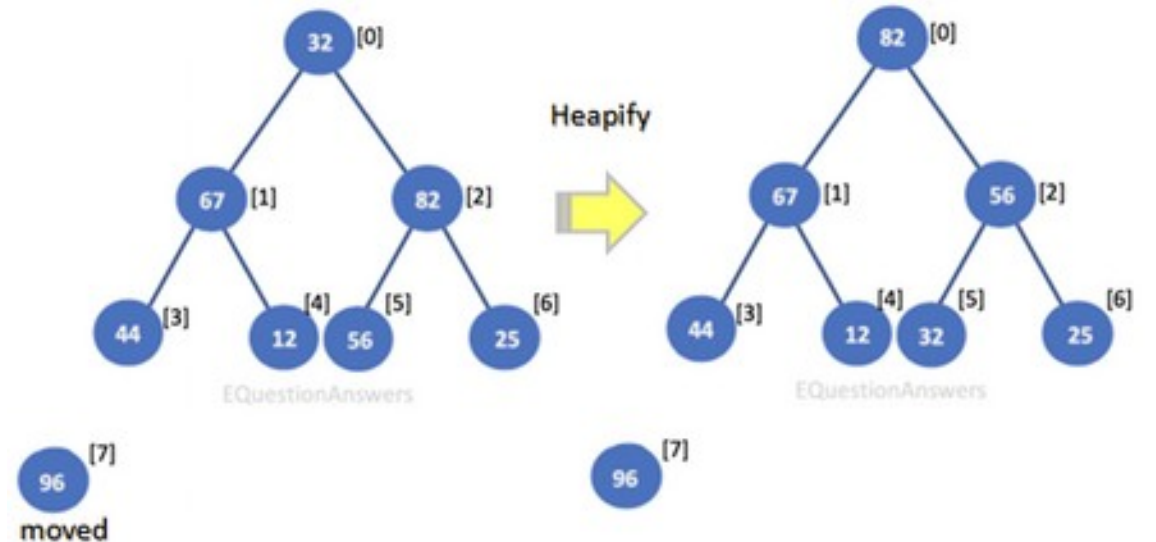
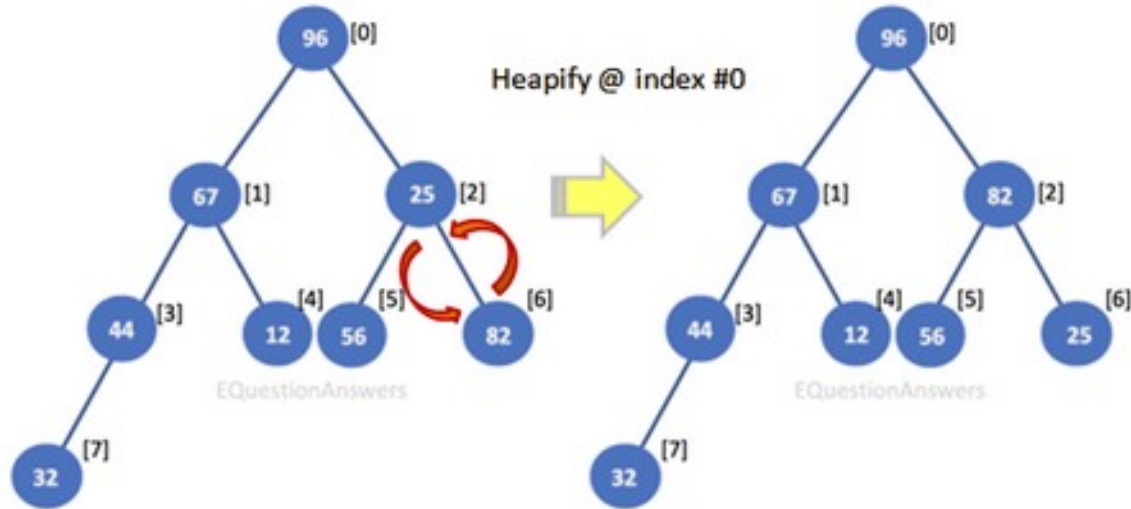
Heap Sort



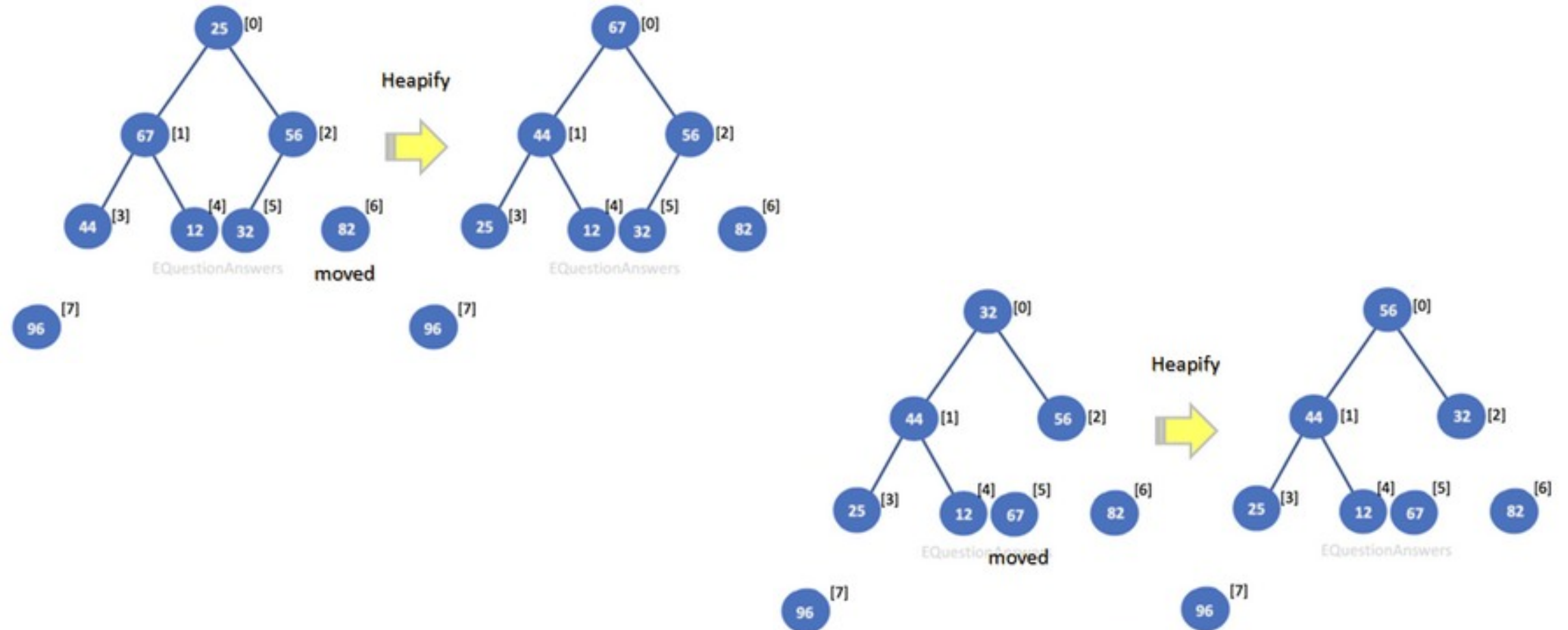
Heap Sort



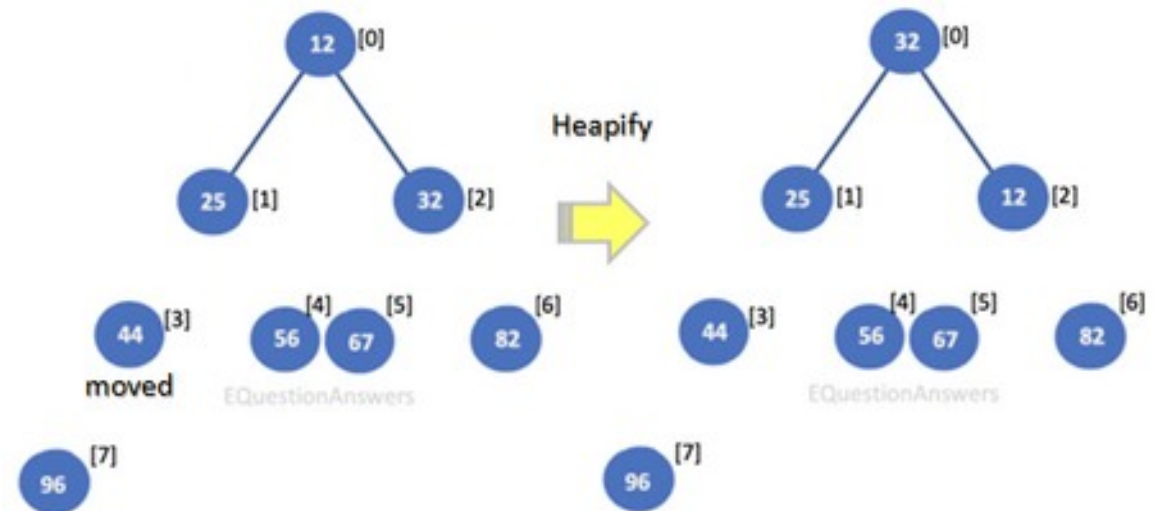
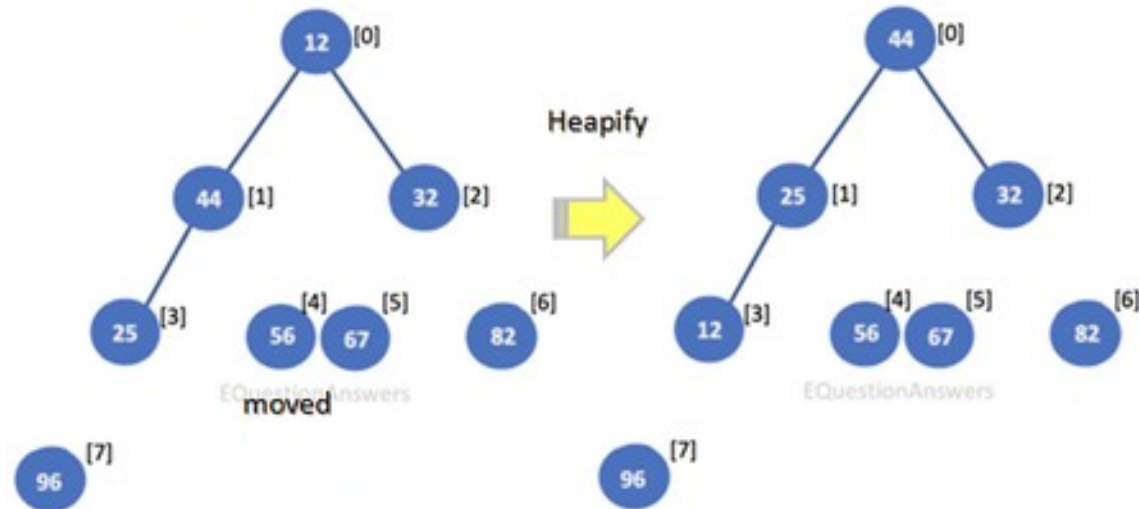
Heap Sort



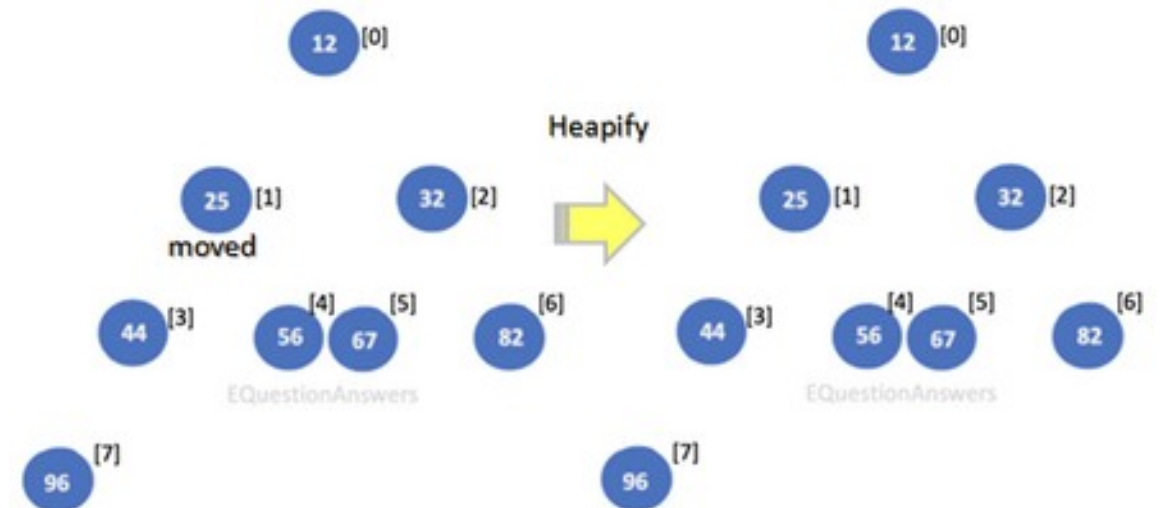
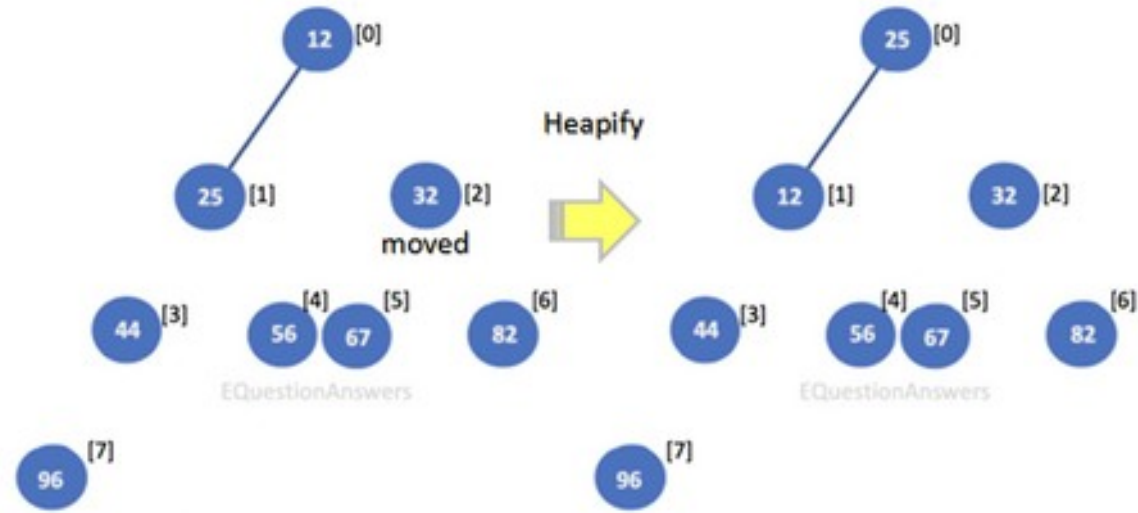
Heap Sort



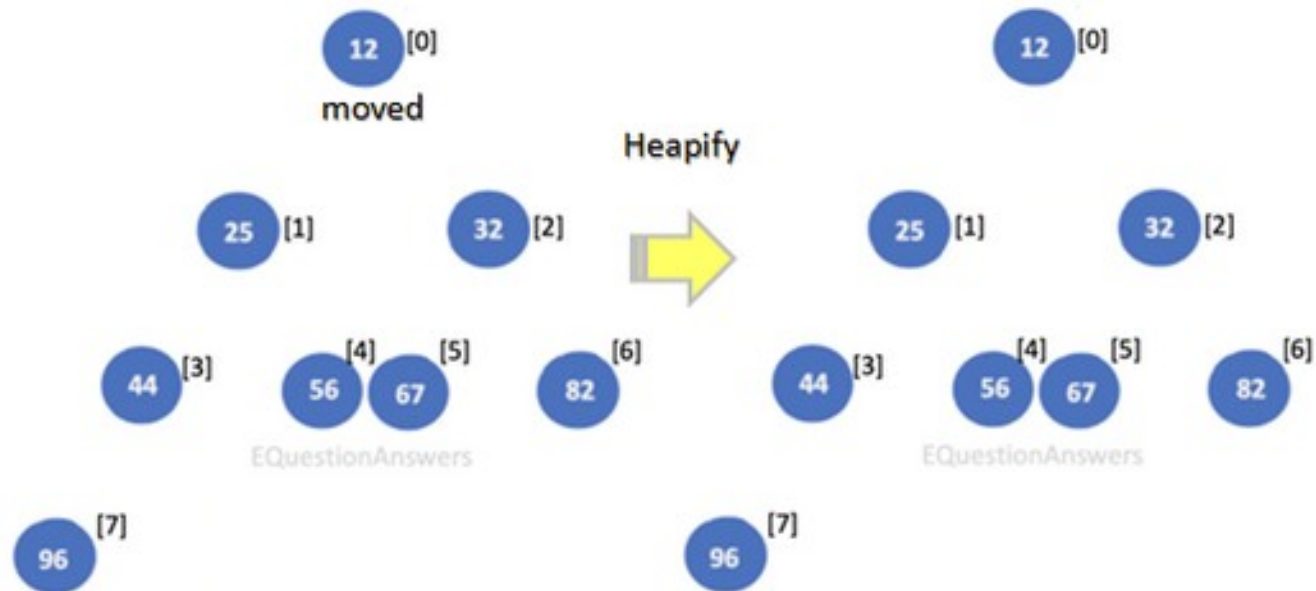
Heap Sort



Heap Sort



Heap Sort



- Sorted array is 12 25 32 44 56 67 82 96

Heap Sort

Heap Sort

```
void heapify(int arr[], int n, int i)
{
    int largest = i; // Initialize largest as root
    int l = 2 * i + 1; // left = 2*i + 1
    int r = 2 * i + 2; // right = 2*i + 2
    // If left child is larger than root
    if (l < n && arr[l] > arr[largest])
        largest = l;
    // If right child is larger than largest so far
    if (r < n && arr[r] > arr[largest])
        largest = r;
    // If largest is not root
    if (largest != i) {
        swap(arr[i], arr[largest]);
        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}
```

Heap Sort

```
// main function to do heap sort
void heapSort(int arr[], int n)
{
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // One by one extract an element from
    heap
    for (int i = n - 1; i > 0; i--) {
        // Move current root to end
        swap(arr[0], arr[i]);

        // call max heapify on the reduced
    heap
        heapify(arr, i, 0);
    }
}
```


Heap Sort

- Time complexity
 - Heapify is $O(\log n)$.
 - createAndBuildHeap() is $O(n)$
 - overall time complexity of Heap Sort is $O(n \log n)$.
- Heap sort is an in-place algorithm.
- Its typical implementation is not stable, but can be made stable

Quick Sort

Algorithm `inPlaceQuickSort( $S, a, b$ ):`

Input: An array, S , of distinct elements; integers a and b

Output: The subarray $S[a .. b]$ arranged in nondecreasing order

if $a \geq b$ then return // subrange with 0 or 1 elements

$l \leftarrow \text{inPlacePartition}(S, a, b)$

`inPlaceQuickSort( $S, a, l - 1$ )`

`inPlaceQuickSort( $S, l + 1, b$ )`

Quick Sort

Algorithm `inPlacePartition( $S, a, b$ ):`

Input: An array, S , of distinct elements; integers a and b such that $a \leq b$

Output: An integer, l , such that the subarray $S[a..b]$ is partitioned into $S[a..l-1]$ and $S[l..b]$ so that every element in $S[a..l-1]$ is less than each element in $S[l..b]$

Let r be a random integer in the range $[a, b]$

Swap $S[r]$ and $S[b]$

$p \leftarrow S[b]$ // the pivot

$l \leftarrow a$ // l will scan rightward

$r \leftarrow b - 1$ // r will scan leftward

while $l \leq r$ **do** // find an element larger than the pivot

while $l \leq r$ and $S[l] \leq p$ **do**

$l \leftarrow l + 1$

while $r \geq l$ and $S[r] \geq p$ **do** // find an element smaller than the pivot

$r \leftarrow r - 1$

if $l < r$ **then**

 Swap $S[l]$ and $S[r]$

Swap $S[l]$ and $S[b]$ // put the pivot into its final place

return l

Quick Sort

- **Worst Case:** The worst case occurs when the partition process always picks greatest or smallest element as pivot. $O(n^2)$
- **Best Case:** The best case occurs when the partition process always picks the middle element as pivot. $O(n \log n)$
- **Average Case:** $O(n \log n)$
- The default implementation is not stable. However, any sorting algorithm can be made stable by considering indexes as comparison parameter.
- in-place sorting algorithm

Quick Sort

Quick Sort

QuickSelect

- A selection algorithm related to Quick sort to find the k -th smallest (or largest) element in an unordered list
- After partitioning around a pivot, the algorithm does not recurse on both subarrays.
- It recurses only on the subarray that may contain the k -th smallest element.
 - If the pivot index $> k$, recur on the left subarray.
 - If the pivot index $= k$, the k -th smallest element is found, so return it.
 - If the pivot index $< k$, recur on the right subarray.
- Because only one side is processed each time, the expected time complexity reduces from $O(n \log n)$ to $O(n)$.
- The worst-case time complexity remains $O(n^2)$ (when partitions are highly unbalanced).

QuickSelect Algorithm

QUICKSELECT(A, low, high, k)

 if low == high

 return A[low]

 p = PARTITION(A, low, high)

 if p == k

 return A[p]

 else if k < p

 return QUICKSELECT(A, low, p-1, k)

 else

 return QUICKSELECT(A, p+1, high, k)

Merge Sort

```
void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    // Create temp arrays
    int L[n1], R[n2];
    // Copy data to temp arrays L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
    // Merge the temp arrays back into
    arr[l..r]
    // Initial index of first subarray
    int i = 0;
    // Initial index of second subarray
    int j = 0;
    // Initial index of merged subarray
    int k = l;
```

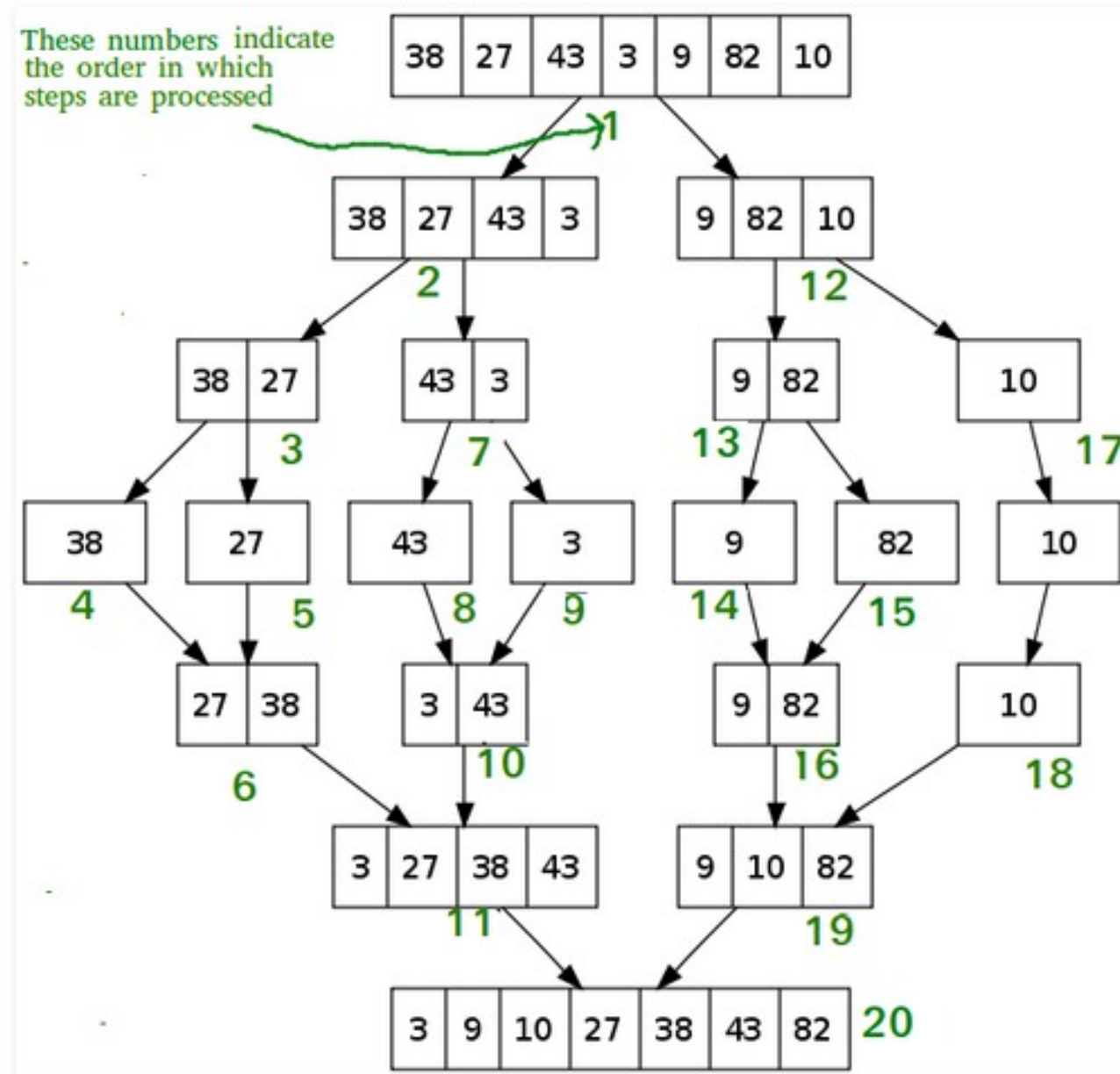
Merge Sort

```
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}
// Copy the remaining elements of L[], if there are any
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}
// Copy the remaining elements of R[], if there
are any
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}
```

Merge Sort

```
void mergeSort(int arr[],int l,int
r){
    if(l>=r){
        return; //returns
recursively
    }
    int m =l+ (r-l)/2;
    mergeSort(arr,l,m);
    mergeSort(arr,m+1,r);
    merge(arr,l,m,r);
}
```

Merge Sort



Merge Sort

- Time Complexity
 - Worst Case Time Complexity [Big-O]: **$O(n \cdot \log n)$**
 - Best Case Time Complexity [Big-omega]: **$O(n \cdot \log n)$**
 - Average Time Complexity [Big-theta]: **$O(n \cdot \log n)$**
- **Algorithmic Paradigm:** Divide and Conquer
- It requires **equal amount of additional space** as the unsorted array. Hence its not at all recommended for searching large unsorted arrays.
- Not In-Place algorithm but stable

IntroSort or Introspective sort

- **Introsort** is a *hybrid comparison-based sorting algorithm*
- Combines:
 - Quicksort
 - Heapsort
 - Insertion Sort
- Designed to:
 - Achieve **fast average-case performance**
 - Guarantee **$O(N \log N)$ worst-case complexity**
 - Maintain **in-place sorting**

Algorithm	Strength	Weakness
Quicksort	Fast average case	$O(N^2)$ worst case
Heapsort	Guaranteed $O(N \log N)$	More swaps
Mergesort	Stable	Not in-place
Insertion	Best for small inputs	Poor scalability

How does IntroSort work?

- Start with **Quicksort**
- If recursion depth exceeds limit → Switch to **Heapsort**
 - $\text{depthLimit} = 2 \times \log(N)$
- If subarray size is small → Use **Insertion Sort**
- Why Not Use Mergesort in Introsort? → requires extra memory

Algorithm	Time Complexity (Worst)	Time (ms)	Stable	In-place
Insertion Sort	$O(N^2)$	High	Yes	Yes
Quicksort	$O(N^2)$	Low	No	Yes
Heapsort	$O(N \log N)$	Medium	No	Yes
Introsort	$O(N \log N)$	Lowest	No	Yes

Counting Sort

- Non-comparison-based sorting algorithm
- Efficient when the input range is small compared to the number of elements.
- Basic idea: Count the frequency of each distinct element in the input array.
- Use the frequency information to determine the correct sorted positions of elements.
- Works well when the range of input values is small and comparable to array size.
 - Example: For input [1, 4, 0, 2, 1, 1]
 - Array size = 6
 - Range of elements = 0 to 4
- If the range of input values is much larger than $n \log n$, where n is array size, It is usually better NOT to use Counting sort. Instead prefer Merge sort or so.

Counting Sort - Algorithm

COUNTING-SORT(A, n)

1. k = maximum element in A
2. Create array $C[0...k]$ and initialize to 0
3. for $i = 0$ to $n-1$
 $C[A[i]] = C[A[i]] + 1$
4. for $i = 1$ to k
 $C[i] = C[i] + C[i-1]$
5. Create array $B[0...n-1]$
6. for $i = n-1$ downto 0
 $B[C[A[i]] - 1] = A[i]$
 $C[A[i]] = C[A[i]] - 1$
7. return B

- **Time Complexity:** $O(N+M)$ in all cases, where N and M are the size of Input and Count Arrays
- **Auxiliary Space:** $O(N+M)$, where N and M are the space of Input and Count Arrays
- **Advantages:**
 - Faster than Merge and Quicksort
 - Stable
- **Disadvantages:**
 - Cannot work for decimal values
 - Inefficient if range is too large
 - Not an in-place algorithm
- **Applications:**
 - Sort students based on grades

Counting Sort - Example

Use counting sort to sort the numbers. Show the array clearly.

- Sort 5, 0, 1, 5, 2, 1, 0, 2, 3, 0
- Sort 4, 2, 2, 8, 3, 3, 1

Bucket sort

130, 413, 224, 106, 923, 456, 321, 003, 623

Radix sort

130, 413, 224, 106, 923, 456, 321, 003, 623

Comparison of Sorting algorithms

Sorting Algorithms						
Algorithm	Best Case (Ω)	Average Case (θ)	Worst Case (O)	array already sorted..	Inplace?	Stable?
Bubble sort	n	n^2	n^2	Best case	Yes	Yes
Selection sort	n^2	n^2	n^2	NIL	Yes	No (at times)
Insertion sort	n	n^2	n^2	Best case	Yes	Yes
Heap sort	$n \log n$	$n \log n$	$n \log n$	NIL	Yes	No
Quick sort	$n \log n$	$n \log n$	n^2	Worst case	Yes	No
Merge sort	$n \log n$	$n \log n$	$n \log n$	NIL	No	Yes
Bucket sort	$n+k$	$n+k$	n^2	NIL	No	Yes
Radix sort	$(n+k)d$	$(n+k)d$	$(n+k)d$	NIL	No	depends on the algorithm
Counting sort	$n+m$	$n+m$	$n+m$	NIL	No	Yes

n – input size, k – buckets, d – maximum digit count in the input number, m – maximum number (range) in the array

Categories

- **Comparison based sorting**
 - Bubble sort
 - Insertion sort
 - Selection sort
 - Merge sort
 - Heap sort
 - Quick sort
- **Non-comparison based sorting**
 - Radix sort
 - Bucket sort
 - Counting sort
- **Internal sorting**
 - Bubble sort
 - Insertion sort
 - Selection sort
 - Heap sort
 - Quick sort
 - Radix sort
 - Bucket sort
 - Counting sort
- **External sorting**
 - Merge sort